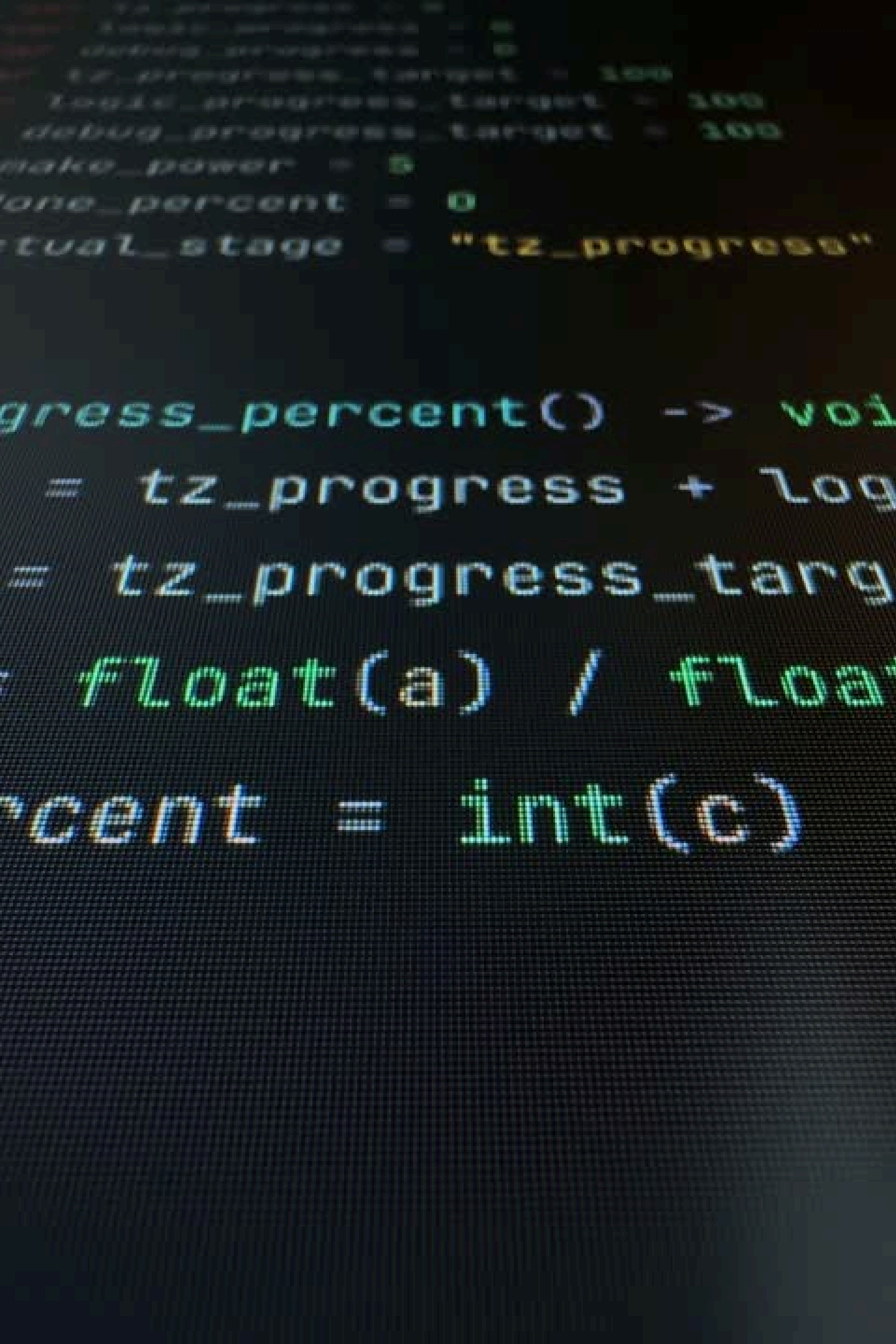




UACAM
Universidad Autónoma de Campeche



UNIVERSIDAD AUTÓNOMA DE CAMPECHE.

Concurrencia y paralelismo.

“Algoritmo de peterson”.

Equipo 2:

1. Aldair Canul Cabrera.
2. Leonardo Gongora Us.
3. Daniel Beytia Chi
4. Emmanuel Cruz Mora.
5. Uriel Quetz Mendoza.
6. Aaron Chi Uc.

```
if ($?) {  
    $distArray = array();  
    $row = mysqli_fetch_assoc($result);  
    $correctAnswer = $row['Correct'];  
    $distArray['A'] = $row['Anum'];  
    $distArray['B'] = $row['Bnum'];  
    $distArray['C'] = $row['Cnum'];  
    $distArray['D'] = $row['Dnum'];  
    $distArray['Correct'] = $correctAnswer;  
    $distArray['Answer'] = rtrim($row[$correctAnswer], ".");  
    $distArray['Query'] = "SELECT * FROM TechTerms WHERE Date='{date}'";  
    return $distArray;  
} else {  
    $distArray['Error'] = 'Quiz load query failed';  
    return $distArray;  
}
```



El problema: la sección crítica

Cuando varios procesos acceden a un **recurso compartido** (memoria, fichero, dispositivo), pueden interferir entre sí y producir **condiciones de carrera**.

Exclusión mutua

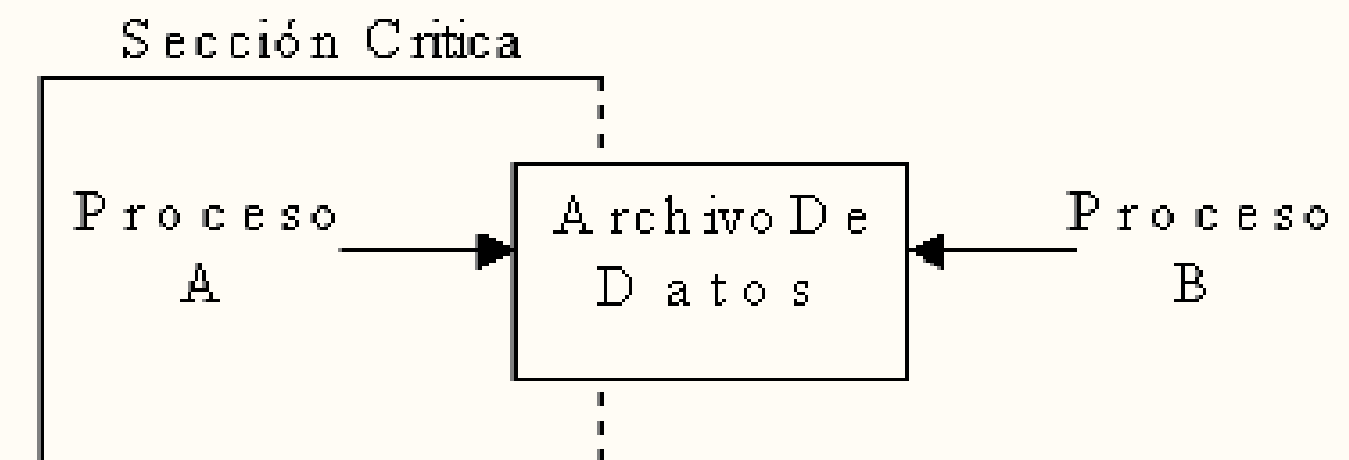
Solo un proceso puede estar en la sección crítica a la vez.

Progreso

Si nadie está en la sección crítica, algún proceso debe poder entrar.

Espera limitada

Ningún proceso espera indefinidamente para entrar (sin inanición).



¿Que es el algoritmo de Peterson?

Algoritmo de programación concurrente para exclusión mutua, que permite a dos procesos/hilos de ejecución compartir un recurso sin conflictos, utilizando sólo memoria compartida para la comunicación.

Peterson usa dos variables compartidas que ambos procesos pueden leer y escribir:

flag[i] — booleana

flag[i] = true indica que el proceso *i* **quiere entrar** en la sección crítica.

turn — entera

Indica a quién le **cede** el turno
(turno actual para esperar/entrar.)

Identificador local (Manera Automatica):

i — identificador

El índice del proceso (0 o 1). El otro proceso es $j = 1 - i$.

Ejemplo:



```
p0: Vegetta777[0] = true
    turno = 1
    while( Willyrex[1] && turno == 1 );
    //{ no hace nada; espera. }
    // sección crítica

    // fin de la sección crítica
    Vegetta777[0] = false
```

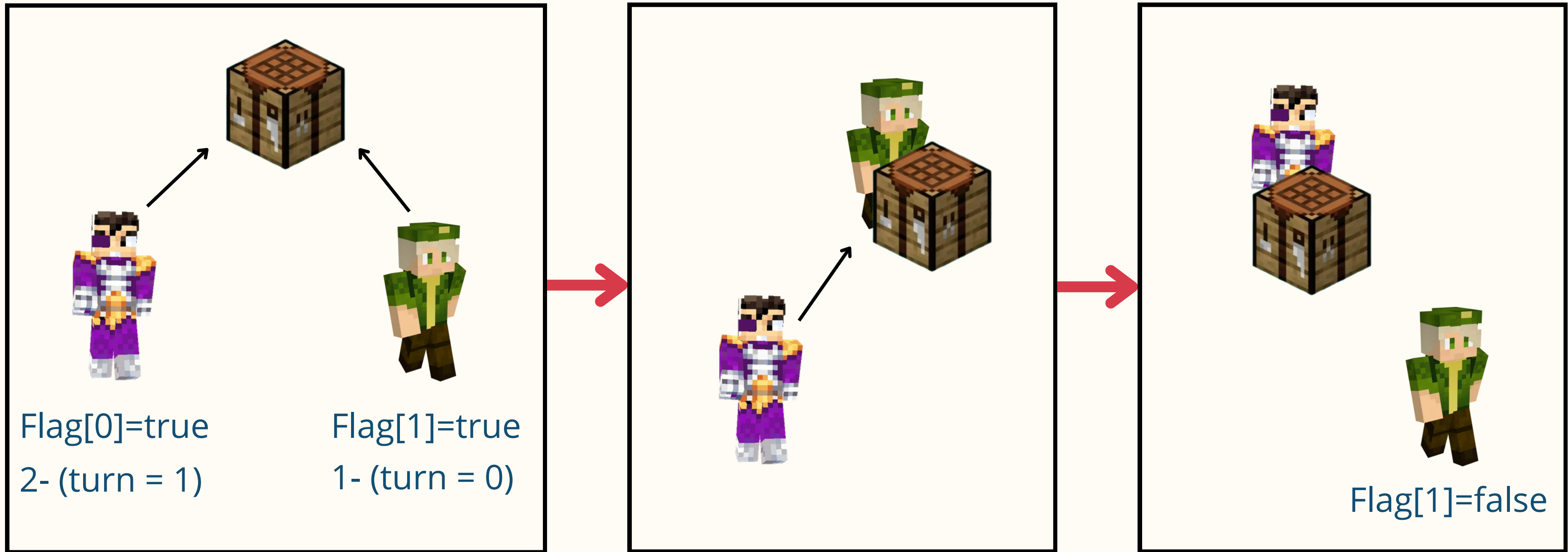
```
p1: Willyrex[1] = true
    turno = 0
    while( Vegetta777[0] && turno == 0 );
    //{ no hace nada; espera. }
    // sección crítica

    // fin de la sección crítica
    Willyrex[1] = false
```

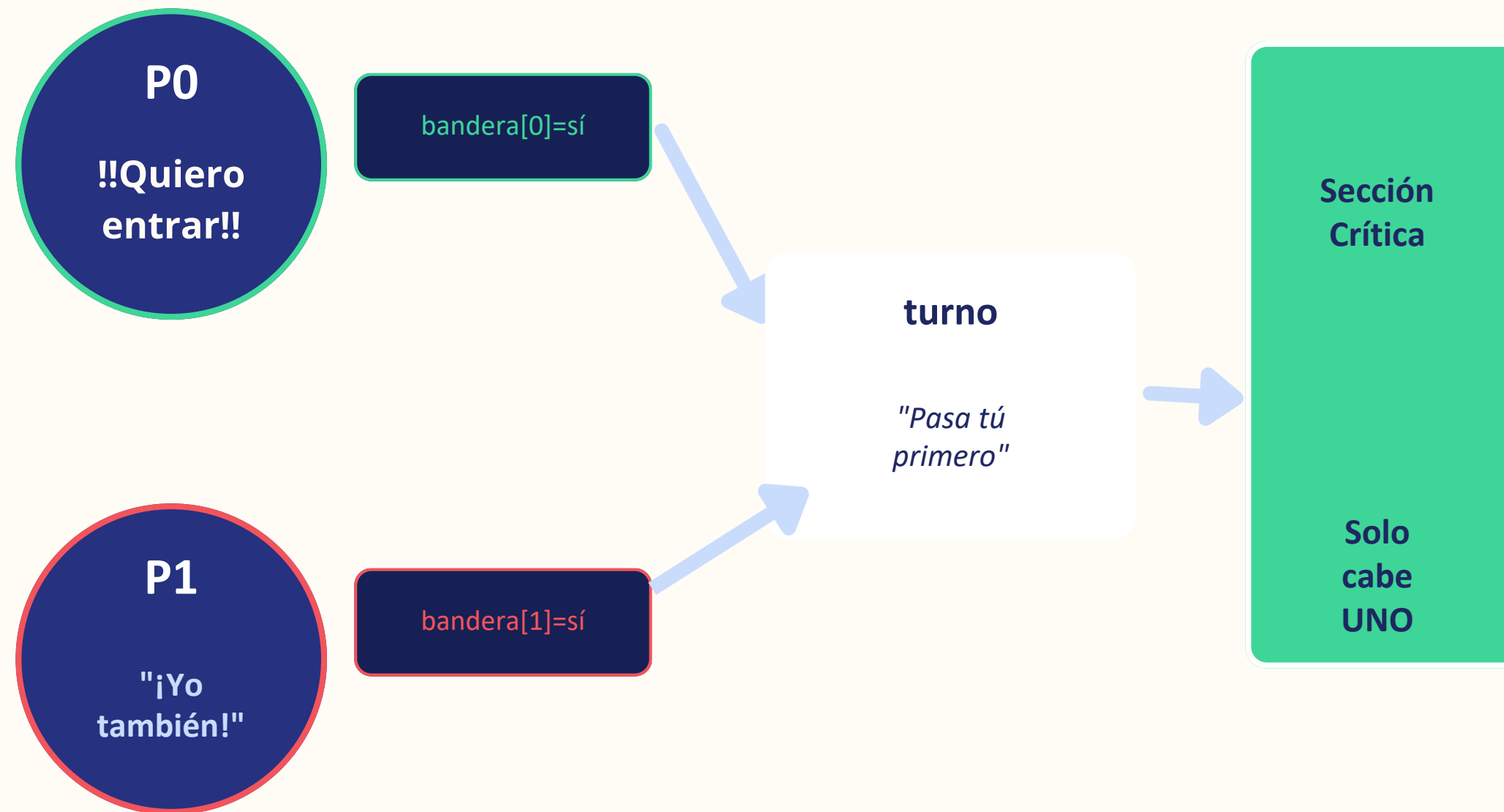


¿Cómo funciona?

"Cortesia"



¿Cómo garantiza el Algoritmo de Peterson la Exclusión Mutua?



1

Levanta la mano

Cada proceso avisa que quiere entrar (pone su bandera en "sí").

2

Cede el turno

Antes de pasar, cada uno le dice al otro: "pasa tú primero".

3

Solo uno gana

Como "turno" solo guarda un valor, si los dos compiten, uno se queda esperando.

4

Nunca hay choque

El que espera solo entra cuando el otro ya salió y bajó su bandera.

Pseudocódigo: estructura del proceso i

```
// Sección de entrada
flag[i] = true
turn = j
while (flag[j] == true AND turn == j):
    esperar (busy-wait)

// SECCIÓN CRÍTICA
//  acceso al recurso compartido

// Sección de salida
flag[i] = false
```

¿Qué ocurre en cada fase?

01

Declarar intención

flag[i] = true

02

Ceder el turno

turn = j

03

Esperar activamente

Mientras el otro quiera entrar y sea su turno.

04

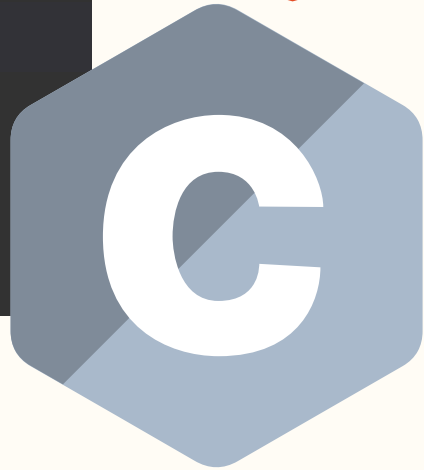
Liberar

flag[i] = false al salir.

Demostración práctica: Peterson en C

```
8
9 // 1. Variables compartidas (Estado global)
10 volatile atomic_int flag[2]; /* flag[i] = 1 indica que el proceso 'i' quiere entrar */
11 volatile atomic_int turno; /* Indica a quién se le cede el paso */
12
13 // 2. Entrada a la Sección Crítica
14 void peterson_lock(int id) {
15     int otro = 1 - id;
16
17     atomic_store(&flag[id], 1); // Levanto mi bandera (tengo intención de entrar)
18     atomic_store(&turno, otro); // Cedo la prioridad al otro proceso (cortesía)
19
20     // Espera activa: me detengo solo si el otro quiere entrar Y tiene el turno
21     while (atomic_load(&flag[otro]) && atomic_load(&turno) == otro) {
22         /* Spin-wait */
23     }
24 }
25
26 // 3. Salida de la Sección Crítica
27 void peterson_unlock(int id) {
28     atomic_store(&flag[id], 0); // Bajo mi bandera (libero el recurso)
29 }
```

HTML



ALGORITMO DE PETERSON Y LA SOLUCIÓN DEL BLOQUEO MUTUO



Ventajas y limitaciones

Ventajas

- Solución puramente software: no requiere instrucciones atómicas de hardware.
- Garantiza los tres requisitos: exclusión, progreso y espera limitada.
- Simple de implementar y verificar formalmente.
- Ideal para enseñanza y sistemas embebidos simples.

Limitaciones

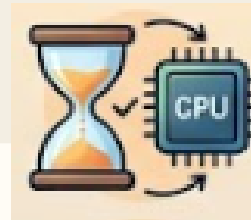
- Solo funciona para **2 procesos**; no escala a N procesos.
- Usa **busy-wait**: consume CPU mientras espera (espera activa).
- No es adecuado para entornos con prioridades o bloqueos reales.
- En arquitecturas con reordenación de memoria puede fallar sin barreras.



Tipos de Limitaciones

Restricción a 2 procesos

Su diseño original solo contempla la competencia entre dos procesos (\$P_0\$ y \$P_1\$).



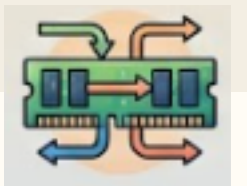
Desperdicio de CPU por Espera Activa

Impacta negativamente en el rendimiento energético y del sistema.



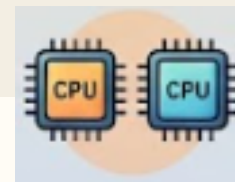
Incompatibilidad con Procesadores

Requiere el uso explícito de barreras de memoria (memory barriers)



Ineficiencia en Arquitecturas M/M

El acceso constante a variables compartidas causa invalidación de caché

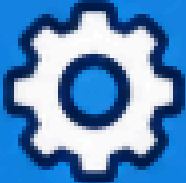
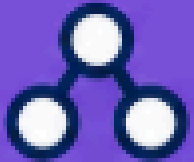
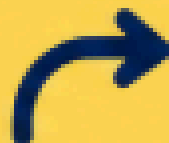


Obsolescencia en Entornos Reales

Si nadie está en la sección crítica, algún proceso debe poder entrar.



Comparativa con otros algoritmos

Aspecto	 Paterson	 Merge Sort	 Quick Sort	 Heap Sort	 Insertion Sort	 Bubble Sort
 Tiempo promedio	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n^2)$
 Peor caso	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(n^2)$	$O(n^2)$
 Memoria adicional	Baja	Alta	Baja	Baja	Baja	Baja
 Estabilidad	Sí	Sí	No	No	Sí	Sí

Cierre y diversión.

Ideas clave para recordar

- Peterson resuelve exclusión mutua para 2 procesos con solo 3 variables compartidas.
- La variable turn es el mecanismo de desempate que evita la carrera.
- Es una solución de software puro, pero no escala y usa busy-wait.

🔔 Porfavor unanse al kahoot